1. Problem Title & Source

Title: **Convert String A to B**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

You are given **two strings**:

- A → the original string
- B → the target string

Your goal is to determine whether it is possible to **convert string A into string B** using a specific rule.

Allowed operation:

- You may **delete any character from string A**.
- However, you **cannot change the order of the remaining characters**.

In other words, you must check whether **string B can be obtained from string A by deleting some characters without rearranging the rest**.

This is essentially checking if **B is a subsequence of A**.

If the conversion is possible, print:

YES

Otherwise print:

NO

3. Examples

Example 1

Input

ABCDEF

ACE

Output

YES

Explanation

From ABCDEF, remove:

B, D, F

Remaining string

ACE

which matches string **B**.

Example 2

Input

ABCDE

AEC

Output

NO

Explanation

Characters of B must appear **in the same order** in A.

Sequence in A:

A B C D E

But B requires:

A E C

which breaks the order rule.

4. Constraints

1 ≤ length(A), length(B) ≤ 100

Observations

- Order of characters must be preserved
- Only deletion allowed

5. Core Concept (Pattern)

Two Pointer / Subsequence Check

Key idea:

Check if B is a subsequence of A

6. Thought Process

Step 1 — Use Two Pointers

Maintain two indices:

i → pointer for string A

j → pointer for string B

Step 2 — Traverse A

If characters match:

A[i] == B[j]

move both pointers.

Otherwise:

move pointer i only

Step 3 — Check Completion

If we reach the end of B:

j == length(B)

then conversion is possible.

7. Visualization

Example

A = ABCDEF

B = ACE

Comparison

A Pointer	B Pointer	Match ?
A	A	Yes
B	C	No
C	C	Yes
D	E	No
E	E	Yes

Result

YES

8. Pseudocode

```
read string A
read string B

i = 0
j = 0

while i < length(A) and j < length(B)

   if A[i] == B[j]
      j++

   i++

if j == length(B)
   print YES
else
   print NO
```

9. Code Implementation

Python

```
A = input().strip()
B = input().strip()

i = 0
j = 0

while i < len(A) and j < len(B):

    if A[i] == B[j]:
        j += 1

    i += 1

if j == len(B):
    print("YES")
else:
    print("NO")
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        String A = sc.next();
        String B = sc.next();

        int i = 0;
        int j = 0;

        while(i < A.length() && j < B.length()){
```

```
            if(A.charAt(i) == B.charAt(j))
                j++;

            i++;
        }

        if(j == B.length())
            System.out.println("YES");
        else
            System.out.println("NO");
    }
}
```

10. Time & Space Complexity

Time Complexity

O(N)

Where

N = length of A

Space Complexity

O(1)

Only pointer variables used.

11. Common Mistakes

1. Trying to **sort strings**
2. Using nested loops unnecessarily
3. Forgetting order constraint
4. Checking substring instead of subsequence

12. Detailed Dry Run

Input

A = ABCDEF

B = ACE

Step	A[i]	B[j]	Action
1	A	A	match
2	B	C	skip

3	C	C	match
4	D	E	skip
5	E	E	match

Since j == len(B)

Output

YES

13. Real-Life Applications

Subsequence checking appears in:

- DNA sequence matching
- Text pattern matching
- Data stream filtering
- Log sequence analysis

14. Common Traps

1. Confusing **substring vs subsequence**
2. Resetting pointer incorrectly
3. Using extra memory
4. Incorrect loop conditions

15. Related Problems

Subsequence problems:

LC 392 Is Subsequence

LC 1143 Longest Common Subsequence

LC 583 Delete Operation for Two Strings

16. Alternate Approaches

Approach	Complexity	Notes
Two pointer	O(N)	optimal
Dynamic programming	O(N×M)	unnecessary

17. Why This Solution Works

Since characters must appear **in the same order**, scanning A once while tracking matches in B guarantees that we correctly verify whether B can be obtained by **deleting characters from A**.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Minimum deletions to convert **A → B**
2. Longest common subsequence
3. Check if two strings are **anagrams**
4. Convert string using **insert/delete operations**